SOFTWARE EVOLUTION | ASSIGNMENT 02

BURHAN'S BUDGET BUDDY

Migration • Layered Refactoring • Class Rewriting

OUR MAIN GOAL

Evolve the existing Budget Buddy application. Do not build an unrelated application from scratch. Preserve Assignment 01 features while demonstrating all three required concepts in working code.

Project context

This plan is based on the Assignment 02 brief and the Budget Buddy project context discussed so far. The expected stack is Python, Streamlit, SQLite, Pandas, Matplotlib and pytest. We will confirm exact filenames, classes, database names and current architecture by inspecting the latest Assignment 01 ZIP before editing.

The three compulsory concepts

Concept	Proposed approach	Proof we will show
A. Migration	Move database configuration into one centralized settings module, provided the current code does not already do this.	Before/after configuration, steps, risk notes, and successful database verification.
B. Refactoring	Refactor Add Expense and Monthly Budget Summary to Controller → Services → Repository → Database.	Architecture diagram and a live trace through all four layers.
C. Rewriting	Rewrite two suitable existing classes/modules; proposed candidates are TransactionValidator and BudgetCalculator.	Old/new design explanation plus unit tests proving behaviour.

Guiding principles

- Protect the Assignment 01 baseline; never make the only copy of the project the working copy.
- Inspect actual code before deciding what to move, refactor or rewrite.
- Keep the existing UI usable and avoid introducing unnecessary tools or complexity.
- Measure real before/after results. Do not claim tests, coverage or improvements that were not measured.
- Both team members should understand the complete design, not only their own files.

Prepared as an implementation guide for the two-member project team.

PART A

Migration: move the system forward

Recommended first choice: centralize database configuration. Confirm the actual starting point during code inspection.

BEFORE → AFTER

Before: database path or settings may be repeated or hard-coded in multiple modules. After: one settings module owns the database configuration, and database-related modules import it.

Why this is a useful migration

- One place to update the database location or environment-specific settings.
- Less duplicated configuration and fewer inconsistent database paths.
- Easier setup for another developer or a different machine.
- A clear, demonstrable change that fits the existing Python and SQLite stack.

Proposed target structure

Add config/settings.py for shared settings and keep database connection setup in the existing database module (for example, data/database.py). Keep the current project folders unless inspection shows a good reason to reorganize them.

Illustrative settings.py

```
import os

# Find the main project directory from this file
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

# Keep the database path in one place
DATABASE_PATH = os.path.join(BASE_DIR, "your_existing_database_file.db")
```

Replace the example database filename with the exact filename used by the current project. Do not create a second, empty database by accident.

Migration procedure

1. Find every database path/connection definition in the current code.
2. Create config/settings.py and move the existing path into it.
3. Update imports, remove duplicate definitions, and search for old paths.
4. Back up the database, run tests and launch the app using the existing data.
5. Compare transaction counts and sample records before and after.

Risks and controls

Risk	Control / verification
Wrong path / empty DB	Back up data; verify the resolved path before launch.
Old path still in use	Search the project and test all DB workflows.
Existing data altered	Compare transaction counts and sample records before/after.

FALLBACK IF THIS MIGRATION ALREADY EXISTS

If configuration is already centralized, select another real migration (for example, a repeatable schema/data migration). Document it separately from the required refactoring.

PART B

Refactoring: required layered architecture

The assignment explicitly requires this dependency direction; do not use MVC as the main architecture.

UI — Streamlit
Forms, buttons, pages, charts and user feedback

CONTROLLER
Receives actions, checks basic input, calls services, returns results

SERVICES
Business rules, calculations, budget logic and operation coordination

REPOSITORY
Database queries and data access only; no business calculations

DATABASE — SQLite
Persistent application records

Required dependency direction: UI → Controller → Services → Repository → Database.

Feature 1: Add Expense

Layer	Responsibility in this feature
Streamlit UI	Collect date, category, amount, type and any other existing required fields.
ExpenseController	Receive form values and perform basic input checks; call the service.
ExpenseService	Apply expense rules and coordinate the operation.
ExpenseRepository	Execute the INSERT query and return a success/failure result.
SQLite	Store the new transaction.

Feature 2: Monthly Budget Summary

Layer	Responsibility in this feature
Streamlit UI	Request and display the selected month and summary values.
BudgetController	Receive the month selection and call the service.
BudgetService	Calculate income, expense totals, remaining budget, category totals and utilization.
BudgetRepository	Fetch only the required transactions/budget records from SQLite.
SQLite	Provide stored records.

Refactoring rules to enforce

- Controllers and services must not execute SQL directly.
- Repositories must not contain business calculations such as remaining-budget formulas.
- Keep the UI usable; it should call controller methods rather than connect to SQLite itself.
- Avoid duplicate logic: shared calculations and validation should have one clear home.
- Keep dependencies pointing downward only: Controller → Services → Repository → Database.

PART C

Rewriting: replace two selected classes

The examples below are candidates. We must inspect the real code and choose classes with demonstrable design problems.

Candidate 1: TransactionValidator

PROBLEM TO LOOK FOR

Rules may be repeated across manual entry and CSV import, or bundled into one hard-to-test method.

New design: one focused class responsible for transaction validation. Split rules into small methods so each rule can be tested independently.

```
class TransactionValidator:
    def validate_amount(self, amount):  # positive, max 2 decimals
        pass
    def validate_category(self, category):  # allowed category
        pass
    def validate_date(self, date_value):  # required format
        pass
    def validate_type(self, transaction_type):  # income/expense
        pass
```

Candidate 2: BudgetCalculator

PROBLEM TO LOOK FOR

Formulas may be duplicated, mixed with database access, or hard to test independently.

New design: a focused calculation class that receives data and returns calculated values. It should not open database connections or execute SQL.

```
class BudgetCalculator:
    def calculate_total_income(self, transactions):  # sum income
        pass
    def calculate_total_expenses(self, transactions):  # sum expenses
        pass
    def calculate_remaining_budget(self, income, expenses):
        pass
    def calculate_budget_percentage(self, expenses, budget):
        pass
```

What makes this a rewrite?

- Replace the selected implementation with a cleaner design; do not merely rename or reformat it.
- Write down the old responsibility and the actual maintenance problem found in the code.
- Define the new methods and expected inputs/outputs before updating dependent code.
- Preserve existing public behaviour where practical; document any intentional interface change.
- Add unit tests for normal cases, invalid inputs and edge cases.

Compatibility and domain rules

Preserve existing transaction rules: positive amounts, at most two decimals, valid categories, YYYY-MM-DD dates where required, and income/expense types. CSV category matching should continue to trim whitespace and ignore case unless an approved change is documented.

IMPORTANT

Choose these candidates only if inspection confirms real problems. Otherwise select existing classes/modules with documented maintenance issues.

QUALITY PLAN

Testing, regression and maintainability evidence

The purpose is to show that the evolved application works and is easier to maintain.

Unit tests

Target	Examples of checks
TransactionValidator	Positive amount; zero/negative amount rejected; decimal precision; valid/invalid category; date format; income/expense type.
BudgetCalculator	Income/expense totals; remaining amount; zero budget; utilization percentage.
ExpenseService	Valid transaction accepted; invalid transaction rejected; service delegates persistence to repository.
BudgetService	Correct monthly totals; category aggregation; service uses repository data.

At least five integration/system tests

ID	Workflow	Expected result
IT-01	Add a valid expense through the UI/controller flow.	Expense is saved and appears in transaction history.
IT-02	Add a valid income transaction.	Income is saved and included in the relevant summary.
IT-03	Open monthly budget summary.	Totals and remaining amount match stored transactions.
IT-04	View category-wise spending.	Category totals match the underlying expense records.
IT-05	Import a valid CSV file.	Valid rows are imported and invalid rows are handled according to existing rules.

Regression testing

Run Assignment 01 tests before and after changes. Recheck adding transactions, history, monthly summaries, category charts and CSV import (where implemented). Record actual pass/fail counts; do not assume prior results still apply.

Maintainability comparison

Indicator	How to measure before and after	Desired direction
Direct SQL in controllers/UI	Search changed controller/UI files for SQL statements or DB connection calls.	Zero direct SQL calls.
Duplicated validation logic	Search for repeated validation conditions and compare the number of distinct copies.	One reusable validation implementation.
Business logic in repositories	Review repository methods for calculations and rules.	Repository handles data access only.
Tests	Record executed/passed/failed test counts using pytest output.	Changed logic covered; no unexplained regressions.

Only report measurements actually collected. If test coverage is reported, record the tool/command used and the exact result. A target is not the same as a measured outcome.

PROJECT MANAGEMENT

Git workflow and two-member task split

Keep the original submission safe and make the work traceable.

Branch strategy

```
main  ------------------------------> Assignment 01 baseline
  \
   `-- assignment-02 -------------------> evolving implementation
```

Create a Git tag or preserve a branch for the Assignment 01 baseline. Do all Assignment 02 work on a separate branch. Make small commits with meaningful messages so the change history shows what changed and why.

Suggested commit sequence

- Create assignment-02 branch and tag/record the Assignment 01 baseline.
- Centralize database configuration (migration).
- Add ExpenseController and ExpenseService; move SQL to ExpenseRepository.
- Refactor Monthly Budget Summary through BudgetController, BudgetService and BudgetRepository.
- Rewrite TransactionValidator and add its unit tests.
- Rewrite BudgetCalculator and add its unit tests.
- Add integration and regression tests; fix defects.
- Update README, change log, diagrams and reports.

Suggested two-member division

Member	Primary responsibility	Shared responsibility
Member 1 (Burhan)	Review baseline; migration; architecture diagram; controller/service/repository refactoring for Add Expense and Monthly Budget Summary.	Review teammate code; run tests; prepare demo.
Member 2 (teammate)	Inspect and rewrite two selected classes/modules; unit tests; integration/system test evidence; regression run.	Review Burhan's code; run tests; prepare demo.

This split is a starting proposal. Adjust it to match the actual codebase and each member's strengths. Both members must be able to explain the whole system and their own commits.

Change log template

Version / commit	Files or classes changed	Concept	Reason and verification
v0.2 / commit ID	settings.py, database.py	Migration	Centralize configuration; verify same DB and records.
v0.3 / commit ID	expense_controller.py, expense_service.py, expense_repository.py	Refactoring	Separate responsibilities; test Add Expense flow.
v0.4 / commit ID	budget_* modules	Refactoring	Separate monthly summary responsibilities; test totals.
v0.5 / commit ID	TransactionValidator, BudgetCalculator	Rewriting	Replace old designs; run unit tests.

EXECUTION CHECKLIST

Roadmap, deliverables and live demonstration

Use this as the team's working checklist from code inspection to submission.

Six-stage implementation roadmap

Stage	Work	Evidence / output
1. Inspect	Open latest Assignment 01 ZIP; map folders, classes and current flows; run baseline tests; save baseline branch/tag.	Code review notes, architecture-before diagram, baseline test output.
2. Migrate	Choose one meaningful migration; implement and verify it.	Before/after, steps, risks, data/config verification.
3. Refactor	Move Add Expense and Monthly Budget Summary through the required layers.	Updated code, architecture-after diagram, live operation trace.
4. Rewrite	Rewrite two justified existing classes/modules; update dependents.	Old/new design notes and unit tests.
5. Test	Run unit, integration and regression tests; fix and record defects.	Test cases, results, defect log, regression evidence.
6. Submit/demo	Measure maintainability, finish reports and ZIP, rehearse demo.	Final package, reports, change log, demo.

Final submission checklist

- Updated source code ZIP, including migration scripts/configuration needed to run it.
- Migration report (approximately 1 page).
- Refactoring report (approximately 1–2 pages) with before/after architecture.
- Rewriting report (approximately 1 page) covering at least two classes/modules.
- Test package: unit tests, at least five integration/system cases, results and regression evidence.
- Before/after maintainability comparison and concise change log.
- Final report of 3–4 pages and a short live demonstration.

Live demonstration order

1. Show the migrated component/configuration and its verification.
2. Trace Add Expense through Controller → Services → Repository → Database.
3. Show the Monthly Budget Summary flow and explain where calculations occur.
4. Show both rewritten classes, the old design problem and new unit tests.
5. Run selected tests and demonstrate existing Assignment 01 features still work.
6. Explain one challenge, one trade-off and one lesson for each concept.

OUR FIRST ACTION

Upload the latest Assignment 01 Budget Buddy ZIP. Before changing code, inspect the real project, run the baseline tests, map the current files and confirm whether the proposed migration and rewrite candidates are appropriate. Then implement one stage at a time.

Planning note: This document is an implementation proposal based on the assignment brief and currently known project stack. Exact filenames, existing class names, database filename, baseline test results and final migration choice must be confirmed from the current project ZIP.